

A GRASP algorithm to solve the unicast set covering problem

Joaquín Bautista*, Jordi Pereira

*Escola Tècnica Superior d'Enginyers Industrials de Barcelona, Universitat Politècnica de Catalunya, Av. Diagonal 647, 7th floor,
08028 Barcelona, Spain*

Available online 19 January 2006

Abstract

The set covering problem (SCP) is a well-known combinatorial optimization problem. This paper presents a GRASP algorithm to solve a special SCP case known in the literature as the unicast set covering problem. The algorithm incorporates a local improvement procedure based on the heuristics to solve binary constraint satisfiability problems (SAT). The quality of the proposed algorithm is tested on a set of reference instances, comparing the obtained results with those found in the literature. Our algorithm improves the best-known solutions for many of these instances.

© 2005 Elsevier Ltd. All rights reserved.

Keywords: Set covering; Optimization; Constraint satisfaction

1. Introduction

1.1. Problem description

The set covering problem (SCP) is a well-known combinatorial optimization problem with numerous applications in such diverse fields as job assignment in manufacturing, selection of operators, simplification of Boolean expressions or service location, see [1,2] between others.

The SCP can be described as follows: given a set M , $|M| = m$ and n subsets $S_j \subseteq M$, $j \in N = \{1, \dots, n\}$ each with a non-negative cost c_j , the objective is to find a minimum cost family of subsets S_j such that each element $i \in M$ belongs to at least one subset of the family. An integer programming formulation of the SCP follows:

$$\begin{aligned} \min \quad & \sum_{j=1}^n c_j \cdot x_j \\ \text{s.t.} \quad & \sum_{j \in N} a_{ij} x_j \geq 1, \quad i \in M, \\ & x_j \in \{0, 1\}, \quad j \in N. \end{aligned}$$

* Corresponding author. Fax: +34 93 401 60 54.

E-mail addresses: joaquin.bautista@upc.edu (J. Bautista), jorge.pereira@upc.edu (J. Pereira).

The variable x_j , equals 1 if the subset S_j is in the selected family, and 0 otherwise. The coefficients a_{ij} take the value 1 when element i belongs to S_j , and 0 otherwise. The matrix $A = (a_{ij})$, $i = 1, \dots, m$, $j = 1, \dots, n$, is known as covering matrix. Each row, the constraints, is associated to an element, while each column, variable, is associated to a subset. We say that S_j covers i or that i is covered by S_j if $a_{ij} = 1$.

A particular case of the problem occurs when the costs, c_j , associated to each subset, S_j , are equal. In this case, all costs may be considered equal to 1. This problem was introduced in [3], and is found under different names in the literature, such as location set covering problem (LSCP), minimum cardinality set covering problem (MCSCP) or unicast set covering problem.

The SCP with arbitrary positive costs and the unicast SCP are NP-hard, see [4], and are therefore considered difficult to solve. This paper presents a GRASP algorithm to solve the unicast SCP. The main difference between our algorithm and previous approaches is the novel use of a local improvement search based on constraint satisfiability problems. The local search procedure allows a better exploration of the solution space and provides a new tool to tackle difficult unicast SCP problems where a local search can be of great importance. To the best of our knowledge, no local search has been proposed for the unicast SCP other than redundancy testing. This is mainly due to the difficulty to define good neighborhoods leading to fast improvements. Our procedure allows searching broad areas with identical objective value. This is achieved by losing the feasibility of the solution. Feasibility can be recovered at any given time while keeping the tentative objective value of the unfeasible solution. This encourages to keep searching high-quality solutions even when the search is working with an unfeasible solution.

After reviewing the available literature about the SCP, we go on to show the existing relation between this problem and constraint satisfiability problems. Finally, the implemented algorithm is presented along with a computational experience with OR-Library instances [5].

1.2. Procedures to solve the set covering problem

The literature covers several exact and heuristic approaches to solve SCPs. We highlight the first successful approaches from Fisher and Kedia, based on a dual heuristic and able to solve instances with up to 200 constraints and 2000 variables [6], and Beasley and Jornsten, also based on a dual heuristic with Gomory cuts and able to solve instances with up to 200 constraints and 4000 variables [7]. More recently, Balas and Carrera have proposed an algorithm based on lagrangean relaxations and subgradient optimization, which clearly outperforms the previous approaches [8].

The heuristic approaches can be divided in two main categories. The first one exploits problem characteristics and specific features of each instance. Examples are lagrangean relaxation-based procedures, subgradient optimization methods [9], the relaxed dual model exploitation [2], and surrogate optimization [10]. The second category includes local search procedures and the adaptation of metaheuristics to the SCP, such as genetic algorithms, [11,12], and ant algorithms [13], as well as specifically tailored local search procedures [14]. The first category has been used in many real-life applications; some with structured data [15], but the quality of metaheuristic approaches using some features from the first category of heuristics, and the late appearance of a highly effective local search procedure make this category a competitive approach.

Due to the unicast version specific characteristics, some specific heuristics have been developed for the unicast case, and some general heuristics have been adapted to the unicast case. See, for instance, Almiñana and Pastor adaptation [16], of a prior Lopes and Lorena proposal [10]. The proposed heuristic, based on lagrangean relaxations and the surrogate problems solution, is tested solving 60 newly generated random instances and five literature-based instances.

We highlight the work of Grossman and Wool [17], comparing several heuristics for the unicast SCP. This work presents a comprehensive comparison of the results offered by a neuronal network, and several heuristics that appeared between 1974 and 1993 for both the unicast and non-unicast problem. The presented heuristics range from greedy ones to rounding algorithms and they are tested using the full set of SCP instances coming from the OR-Library, indicating that the greedy heuristic and two its variants are the best suited to solve the problem. Even if several heuristics are presented, no local search procedure is implemented or tested other than redundancy testing, mostly due to the lack of effective local search algorithms for the unicast SCP. The focus of this work is to present a new algorithm to solve the unicast SCP and to develop a new local search for the problem.

2. The set covering problem as a constraint satisfiability problem

2.1. Transformation of the problem into a of maximum constraint satisfiability problem

The relation between the SCP and the maximum satisfiability problem is well-known, as noted in Garey and Johnson [4] and Papadimitriou [18], where the relationship between both problems is put forward. To the best of our knowledge this relationship has not been used to develop algorithms to solve the unicast case, even if at least one paper has used arbitrary cost set covering instances to evaluate the quality of the maximum satisfiability algorithms [19].

The constraint satisfiability problem consists of assigning a true or false value to a set of literals in such a way that it satisfies a clause set in normal disjunctive form.

If the goal consists in searching a solution satisfying all clauses, we have a feasibility problem known as SAT. If the goal is to satisfy the maximum number of clauses, we have an optimization problem known as MAX SAT.

A simple scheme to transform the SCP into a constraint satisfiability problem follows:

- Create a set W of n literals. Each literal w_j is associated to one subset $S_j \subseteq M$, $j = 1, \dots, n$. A literal w_j takes value “true”, or “false”, if the variable x_j associated to the set S_j is 1 or 0, respectively.
- Define a set of unitary clauses Z corresponding to the complement of each literal of W . $Z = \{z_j = \neg w_j : w_j \in W\}$
- Define a set of clauses Y , made up of m clauses, each one associated with an element of the covering problem. Each clause is formed by the disjunction of W literals whose original subsets are said to cover the element in the covering problem.

The resulting instance has n literals and $n + m$ clauses, the goal being to satisfy the maximum number of clauses of the set $Y \cup Z$. Algorithm 1, presented below, may be used to transform a solution obtained for the associated MAX SAT instance into a SCP solution. The algorithm uses as input the subset W_s of clauses from W with true assignment and the subsets of literals Y_s and Z_s with true value. The later can be derived from W_s but is considered an input for convenience.

Note that transforming a SCP solution to an associated MAX SAT problem solution is straightforward.

Algorithm 1: Transformation of a MAXSAT solution into SCP solution

Input: $Y_s = \{y_i \in Y : y_i = \text{true}\}$; $W_s = \{w_j \in W : w_j = \text{true}\}$; $Z_s = \{z_j \in Z : z_j = \text{true}\}$

Output: SCP Solution, with value $|W_s|$ and variables $x_j = 1$, $j \in N$, when $w_j \in W_s$.

1. Provided $|Y_s| = m$, W_s can be mapped to a solution of the associated SCP, with value $|W_s|$. If $|Y_s| < m$, continue.

2. $\forall z_j \in Z_s$, find:

$Y(z_j) = \{y_i \in Y \setminus Y_s : z_j = \text{false} \Rightarrow y_i = \text{true}\}$

Determine $z^* : |Y(z^*)| = \max_{z_j \in Z_s} \{|Y(z_j)|\}$

Continue:

$z^* = \text{false}$

$Z_s = Z_s \setminus \{z^*\}$

$W_s = W_s \cup \{w^*\}$, with $w^* = \neg z^*$

$Y_s = Y_s \cup Y(z^*)$

3. Return to 1

An example of the transformation procedure is given below.

Let the following example be the mathematical formulation of a covering problem with four variables and three constraints:

$$\begin{aligned} \min \quad & x_1 + x_2 + x_3 + x_4 \\ \text{s.t.} \quad & \\ & x_1 + x_2 \geq 1, \\ & x_2 + x_3 \geq 1, \\ & x_3 + x_4 \geq 1. \end{aligned}$$

The set W of literals belonging to the associated satisfiability problem is $W = \{w_1; w_2; w_3; w_4\}$. The set of clauses Z would be formed by four clauses $Z = \{\neg w_1, \neg w_2; \neg w_3, \neg w_4\}$ and the set of clauses Y would be formed by three clauses $Y = \{w_1 \vee w_2; w_2 \vee w_3; w_3 \vee w_4\}$. An optimum solution to the MAX SAT problem is $w_1 = \text{false}$, $w_2 = \text{true}$, $w_3 = \text{true}$, $w_4 = \text{false}$, which satisfies all the clauses of set Y , and two of the clauses of set Z . The associated covering problem solution is $x_2 = x_3 = 1$, with a value of 2, and obviously, it is the optimal solution for the SCP instance.

2.2. Procedures to solve SAT binary satisfiability problems

The literature reports diverse heuristic procedures to solve SAT and MAX SAT problems.

The heuristics for SAT problems are usually based on the iterative application of two different phases. The first phase consists of a constructive procedure allowing an initial solution to be obtained using some heuristic rule or a lot-drawing procedure. The proposed procedures for this first phase range from GRASP procedures, mostly found in MAX SAT [20,21], to random assignments, see [22], more likely found in SAT.

Once obtained a solution to the problem, the second phase, known as local search, seeks solutions of higher quality until it reaches a limited number of attempts or it does not find any better solution. The local search takes the initial solution and searches its neighborhood, choosing one of them to become the new current solution. This search is conducted through a neighborhood relation specifying the possible solutions that can be reached in one step of the local search for each solution. Most of the conducted research is associated to this phase, and a review of different procedures and a quantitative evaluation of their performance is to be seen in [23].

Both phases are repeated during a specified number of iterations or until a stopping criterion is met. Optionally, an information extraction phase may also be used. This phase analyzes the solution obtained and modifies the search space according to it. An example can be seen in the guided local search heuristic [24].

Among the procedures developed for the second phase, local search, we may highlight the GSAT [25], and WALKSAT procedures [26]. Both procedures have reported good results for several feasibility problems sets. They may be seen as descent algorithms with some diversification to avoid getting trapped in local optima. Starting from a random assignment to each literal, they try to increment the number of satisfied clauses until a feasible assignment is found or a number of iterations is reached.

GSAT counts on a neighborhood relation based on FLIP exchanges. A FLIP exchange consists in negating the assignment of a single literal. During each iteration of the GSAT local search, all possible FLIP exchanges for the current solution are tested and the best one is implemented, chosen randomly in case of a tie.

WALKSAT adds a mechanism to the GSAT procedure which allows movements that worsen the quality of the incumbent solution, as in Tabu search [27] though in a much simpler format. The mechanism consists of including a certain level of random search. With high probability each iteration will use the GSAT rule. If not, a random literal is chosen and flipped.

To avoid stagnation in local optimums, the constructive procedure is used as a diversification mechanism. Provided that GSAT and WALKSAT construction procedures are based on random assignments to literals, the search is restarted from a very different solution.

The present study implements the WALKSAT local improvement within a GRASP schema, substituting the random generation of initial solutions by a generation phase that builds initial solutions sequentially, taking into account information about the most promising literals to appear in the final solution.

3. A GRASP algorithm for the problem

3.1. The GRASP metaheuristic

The GRASP metaheuristic [28] is a random iterative optimization procedure. This metaheuristic has been used to solve diverse problems of optimization, including scheduling, route design, logic, location, graphs, assignment, manufacturing, transport, and telecommunications problems, among others. A review of the literature and applications of the GRASP algorithm can be found in [15].

Each iteration in the metaheuristic is made up of two phases: a constructive and a local search phase. During the constructive phase, the algorithm uses a randomized greedy heuristic to obtain an initial solution to the problem. This

is based on modified greedy procedures, where the greedy rule is substituted by a random selection among a limited list of candidates showing the best values for the greedy selection rule.

On the other hand, the local search phase permits exploration of the generated solution neighborhood in an attempt to find higher-quality solutions. After local search, the best solution found during this phase is compared to the best-known solution, and substitutes it if the objective value is better than the previously known. Once a stopping criterion is met, the best solution obtained during the procedure is returned.

In order to solve an optimization problem by means of a GRASP procedure, it is necessary to define at least the following elements integrated in the heuristics:

- the randomized constructive procedure used during this procedure,
- the neighborhood of the solution and the procedure to investigate it,
- the stopping criterion, usually associated to a maximum number of iterations.

Algorithm 2 shows the general GRASP metaheuristic scheme. Note that this heuristic may be extended using post-processing elements of the generated solutions [29] or parameters auto-adaptation during the search [30].

Algorithm 2: Main block of the GRASP metaheuristic.

INPUT: seed, number_iterations

OUTPUT: Solution

1. Initialization.

2. For all $k, 1 \leq k \leq \text{number_iterations}$

a. Solution $\leftarrow$ Constructive_phase(seed)

b. Solution $\leftarrow$ Local_improvement(Solution, seed)

c. Update_solution(Solution, Best_solution)

3. End

One of the major advantages of the GRASP metaheuristic is how easy this general scheme may be adapted to the solution of particular problems. GRASP requires few parameters, basically the stopping criterion, associated to the maximum number of iterations, and a rule to construct the restricted candidate list (RCL) during the constructive phase. Additionally, Algorithm 2 also uses a seed for random number generation. This seed is used during the randomized greedy heuristic and allows the subsequent replication of any run. We keep track of the best solution obtained during the procedure initializing its objective to a high value.

3.2. Constructive phase

Procedure 1 illustrates the constructive phase used in algorithm 2, Step 2a. The procedure starts with an unfeasible trivial solution $w_j = \text{false}, \forall j = 1, \dots, n$. At each iteration of the construction phase, a literal is selected and its value is flipped to true until a feasible solution regarding each clause from Y is obtained.

The selection of the next literal to flip is limited to a RCL which consists of the most promising literals according to a desirable function. The desirability of each literal equals the number of additional satisfied clauses from Y obtained by a true assignment to the literal. The RCL is made up of literals whose desirability is higher than a threshold value L . This threshold is based on the desirability of the best candidate and deterioration parameter $\alpha \in [0, 1]$. When $\alpha = 0$, the constructive algorithm corresponds to a totally random algorithm; when $\alpha = 1$, the algorithm corresponds to a deterministic constructive algorithm with random resolution of ties.

Once the RCL is available, a candidate is then randomly chosen from the restricted list.

Procedure 1: Constructive_phase(seed)

Input: seed for random number generator

Output: A MAXSAT solution defined by the sets W_S, Y_S, Z_S

1. $W_S = \emptyset$

2. $Y_S = \emptyset; Z_S = Z$

3. While $|Y_S| < m$ do :
 - a. $\forall z_j \in Z_S$, find: $Y(z_j) = \{y_i \in Y \setminus Y_S : z_j = \text{false} \Rightarrow y_i = \text{true}\}$
 - b. Determine limit L
 $L = \alpha \cdot \max_{z_j \in Z_S} \{ |Y(z_j)| \}$
 - c. Create candidate list
 $RCL = \{ z_j \in Z_S : |Y(z_j)| \geq L \}$
 - d. Selection of candidate
 $z^* = \text{RND}(RCL, \text{seed})$
 - e. Update
 $z^* = \text{false}$
 $Z_S = Z_S \setminus \{z^*\}$
 $W_S = W_S \cup \{w^*\}$, with $w^* = \neg z^*$
 $Y_S = Y_S \cup Y(z^*)$
4. End

The constructive procedure shown above is based on a greedy SCP heuristic. At every iteration the procedure chooses a literal associated to a variable which is not yet present in the solution. Literals are evaluated according to the number of unsatisfied Y clauses where the literal is present, equivalent to the number of additional constraints satisfied by the related variable in the SCP. The procedure differs from the greedy procedure because the literal is chosen with equal probabilities between a RCL using the RND function.

Even if the procedure is not valid for all the satisfiability problems, it uses some of the covering problems specific characteristics.

3.3. Improvement phase

The solutions generated by the constructive procedure are not necessarily optimal, even with respect to simple neighborhoods. Therefore, a local search phase usually improves the solutions provided by the constructive phase. The expected effectiveness of the local search procedure depends on a variety of aspects, such as the structure of the neighborhood definition, the neighborhood search techniques, the evaluation of the neighbors cost function, the explored neighborhood, and the initial solution.

Procedure 2 shows the WALKSAT improvement procedure used in this paper [26]. The procedure applies a descent procedure with probability p , choosing the best neighboring solution from the initial reachable solution applying the best available flip exchange, $\text{BestFlip}()$. Otherwise the procedure applies an exchange to escape from local optimums, $\text{RandomFlip}()$, which consists of randomly choosing the literal to flip and applying the exchange.

Procedure 2: $\text{Local_improvement}(\text{Solution}, \text{seed})$

INPUT: solution, random seed

OUTPUT: Best_solution

1. Best_solution $\leftarrow$ solution
2. For $i = 1$ to MAXFLIPS
 - a. If $\text{Random}[0, 1] < p$ then
 - i. Solution $\leftarrow$ BestFlip(Solution)
 - b. Else
 - i. Solution $\leftarrow$ RandomFlip(Solution, seed)
 - c. End If
 - d. Update_solution(Solution, Best_solution)
3. End For

While the constructive phase uses a procedure in consonance with the procedures to solve the SCP, the local search phase is completely based on the constraint satisfiability problem. During local search phases, the solution may therefore be no longer feasible for the original covering problem, as it may not fulfill constraint-associated clauses. In exchange, the procedure allows the exploration of feasible solutions areas unreachable without an unfeasibility phase. In the case of the procedure returning an unfeasible solution, Algorithm 1 may be applied to achieve a feasible solution.

4. Computational results

The algorithm was programmed in C and compiled using GCC 3.2, with the $-O3$ optimization flag. All the runs were carried out on a Pentium 4 computer at 1800 MHz with 512 Mb RAM under the Linux operating system. The test instances are described in Tables 1 and 2. They were obtained from the instance sets for the general SCP available from the OR-Library [5]; as in previous studies devoted to the unicast problem, the costs associated to the variables appearing in each instance were ignored.

The control parameters of the GRASP heuristic and WALKSAT local search were fixed to $\text{number_iterations} = 500$, $\alpha = 0.9$, $\text{MAXFLIPS} = 10 * |W|$ and $p = 0.75$ for the computational experience.

Obviously the number of iterations, equal to the number of solutions generated during the construction phase, and number of flips, equal to the number of different neighborhoods explored during each local search phase, is related to the computation time, although an increase does not necessarily improve the quality of the obtained solution. Fig. 1 shows, for the instances of the set 4, the evolution of the solutions when the number of iterations increases. It can be seen that once a certain number of iterations has been reached, the average value of the obtained solutions, as well as the value of the final best solution does not change, and neither does the value of the final best solution.

In order to evaluate the quality of the solution obtained by the GRASP heuristic presented here, we compare the solution offered by the algorithm with the value of the integer linear program relaxation and the best solution reported in Grossman and Wool [17].

Table 1
Test problems features

Set	Rows (m)	Columns (n)	Density (%)	Maximum no. of 1's per row	No. of instances	Problem type
4	200	1000	2	36	10	Random
5	200	2000	2	60	10	Random
6	200	1000	5	71	5	Random
A	300	3000	2	81	5	Random
B	300	3000	5	192	5	Random
C	400	4000	2	105	5	Random
D	400	4000	5	244	5	Random
E	50	500	20	124	5	Random
NRE	500	5000	10	561	5	Random
NRF	500	5000	20	1086	5	Random

The density is the percentage of 1's appearing in the A matrix, while the maximum number of 1's appearing in any constraint is indicated by the maximum number of 1's per row.

Table 2
Detail of the individual instances used in the computational experience

Instance	Rows (m)	Columns (n)	Density (%)	Min–max 1's per row	Problem type
CYC.6	240	192	2.1	4–4	Logical
CYC.7	672	448	0.9	4–4	Logical
CYC.8	1792	1024	0.4	4–4	Logical
CYC.9	4608	2304	0.2	4–4	Logical
CYC.10	11 520	5120	0.08	4–4	Logical
CYC.11	28 160	11 264	0.02	4–4	Logical
CLR.10-4	511	210	12.3	10–126	Combinatorial
CLR.11-4	1023	330	12.4	20–210	Combinatorial
CLR.12-4	2047	495	12.5	30–330	Combinatorial
CLR.13-4	4095	715	12.5	50–495	Combinatorial

For each instance, the number of rows, columns, density and minimum and maximum number of 1's per row of the matrix A are given. The origin of the instance is also provided [17].

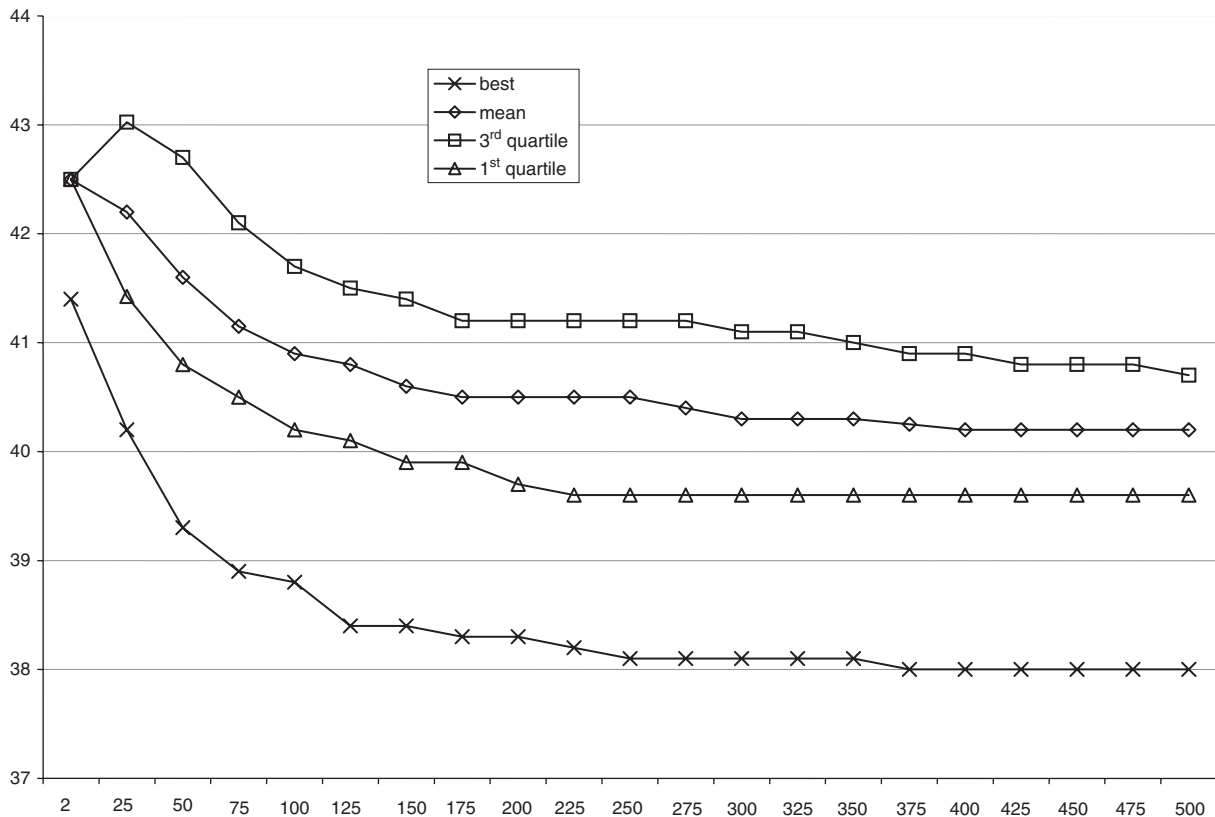


Fig. 1. Evolution of the best solution obtained according to the number of solutions generated by the instances of the set 4. The horizontal axis indicates the number of iterations, while the vertical axis represents the mean objective value obtained for the ten instances of set 4.

It can be seen that for 47 of the 70 instances, our algorithm improves the solutions previously reported in the literature and only in one case the solution obtained is worse than the previously provided. This is mainly due to the improvement provided by a local search procedure designed to solve unicast problems (as is the case of SAT satisfiability problems). Unfortunately, the local search procedure comes with a high computational cost, making it difficult to compare the quality of the GRASP solutions with the previously reported solutions. (Table 3)

This remark, led us to develop a final experiment to compare the best performing heuristics of Grossman and Wool and our proposal in other terms than just quality solution.

We chose the best performing heuristic from the Grossman and Wool experiment, R-Gr, to compare its performance with the constructive phase of the GRASP heuristic and the total GRASP heuristic. R-Gr is a randomized greedy algorithm. In each iteration of the construction phase the variable that appears in the largest number of unsatisfied inequalities is picked. Ties are broken at random. A post optimization procedure is also used for each generated solution, based on a redundancy elimination procedure [17]. Grossman and Wool iterated the basic algorithm for 100 runs as ties are very common and they vary greatly the performance of the algorithm.

The R-Gr algorithm can be seen as a special GRASP algorithm with $\alpha = 1$ and redundancy elimination procedure as the local search phase. We implemented the redundancy elimination procedure and iterated the algorithm for 100 000 iterations. Even if the running times were still far smaller than the proposed algorithm, we stopped the search, as no improvement for any instance was found during the last 10 000 iterations.

Table 4 compares the R-Gr algorithm, GRASP with $\alpha = 0.9$ and redundancy elimination, GRASP without WALKSAT and the solution found by the GRASP with WALKSAT. The GRASP algorithm with WALKSAT local search outperforms all other algorithms obtaining the best solutions for each instance and improving any other algorithm in 13 instances, but showing a larger computational running time. Let us note that in most cases the algorithm does not require the given running to find the reported solutions. The results for the C and D sets were found within

Table 3
Results from the computational experience for each instance

Inst.	LP	Best	Best GRASP	Inst.	LP	Best	Best GRASP
4.1	32.80	41	38	C.1	31.10	45	43
4.2	31.71	38	37	C.2	31.42	45	44
4.3	32.45	40	38	C.3	31.05	45	44
4.4	33.27	41	39	C.4	31.04	46	44
4.5	32.79	40	38	C.5	31.28	45	44
4.6	32.24	40	38	D.1	14.58	26	25
4.7	33.52	41	38	D.2	14.74	25	25
4.8	31.77	40	38	D.3	14.65	25	25
4.9	32.89	40	38	D.4	14.63	26	25
4.10	33.31	41	38	D.5	14.62	26	25
5.1	28.73	35	35	E.1	3.48	5	5
5.2	28.58	35	34	E.2	3.38	5	5
5.3	28.62	36	35	E.3	3.30	5	5
5.4	28.61	36	34	E.4	3.45	5	5
5.5	28.25	36	34	E.5	3.39	5	5
5.6	28.86	36	34	NRE.1	8.18	17	17
5.7	28.09	36	34	NRE.2	8.18	17	17
5.8	28.93	37	35	NRE.3	8.19	17	17
5.9	28.98	36	36	NRE.4	8.19	17	17
5.10	28.79	36	35	NRE.5	8.19	17	17
6.1	14.78	21	21	NRF.1	4.36	10	10
6.2	14.29	21	20	NRF.2	4.36	11	10
6.3	14.87	21	21	NRF.3	4.36	11	10
6.4	14.68	22	21	NRF.4	4.36	11	10
6.5	14.92	22	21	NRF.5	4.36	11	10
A.1	29.55	40	39	CYC.6	48.0	60	60
A.2	29.82	41	39	CYC.7	112.0	144	144
A.3	29.76	40	39	CYC.8	256.0	352	348
A.4	29.09	40	38	CYC.9	576.0	816	813
A.5	29.57	40	39	CYC.10	1280.0	1916	1918
B.1	14.09	23	22	CYC.11	2816.0	4268	4268
B.2	13.99	22	22	CLR.10-4	21.00	28	25
B.3	14.06	22	22	CLR.11-4	16.50	27	23
B.4	14.01	23	22	CLR.12-4	16.50	27	23
B.5	13.98	23	22	CLR.13-4	16.50	29	23

LP is the value obtained by linear relaxation of the problem; “Best”, the best solution reported in Grossman and Wool [17]; “Best GRASP”, the best solution found for the proposed algorithm with number_iterations = 500, $\alpha = 0.9$, MAXFLIPS = $10^*|W|$ and $p = 0.75$. The best result for each instance is reported in boldface.

comparable running times to those reported by the R-Gr heuristic, and NRE and NRF instances were solved in a few seconds.

The second best performing heuristic is the GRASP algorithm without WALKSAT search. This algorithm uses the redundancy test as the local search phase of the GRASP heuristic and obtains the best-known solution for 51 out of 70 instances, with a minimal running time increase compared to the R-Gr heuristic, which is the fastest algorithm.

From the previous results, we conclude that the algorithm to apply depends on the available time to solve the instance. When the running time is not an issue, our WALKSAT algorithm can be applied as it is capable of improving the solution of several instances, while if the available time is an issue and a fast result must be provided, the GRASP approach with redundancy testing is preferable.

Let us note that the different characteristics of the instances modify the quality of the solutions found from our proposal. When the density of the matrix A is relatively small, as instance sets 4, 5, A and C, the WALKSAT search usually obtains better solutions than the construction procedures with redundancy test. When the density of the matrix is high, the local search phase does not improve the quality of the solution, mostly because the quality of the solutions obtained by the constructive procedure plus redundancy testing are very near to the optimal or even optimal.

Table 4
Results for each instance

Inst.	R-Gr 10 ³ it.	R-Gr 10 ⁵ it.	GRASP 10 ³ it.	GRASP 10 ⁵ it.	GRASP +SAT	R-Gr (s)	GRASP (s)	GRASP+ SAT (s)
4.1	40	39	40	39	38	39	59	85
4.2	38	37	37	37	37	37	56	86
4.3	40	39	40	39	38	39	60	85
4.4	40	40	41	40	39	40	60	87
4.5	40	39	39	39	38	37	58	87
4.6	39	38	39	38	38	39	60	86
4.7	40	39	41	38	38	40	60	84
4.8	39	38	39	38	38	38	57	89
4.9	40	39	40	39	38	40	60	88
4.10	40	40	40	39	38	39	59	86
5.1	36	35	35	35	35	55	92	313
5.2	35	34	35	34	34	54	91	319
5.3	35	35	35	35	35	53	93	316
5.4	35	35	35	34	34	53	92	322
5.5	35	35	35	35	34	54	94	327
5.6	35	35	36	35	34	54	95	329
5.7	35	35	35	34	34	53	95	315
5.8	37	36	37	35	35	54	97	314
5.9	36	36	36	36	36	55	96	313
5.10	36	35	35	35	35	55	94	308
6.1	22	22	21	21	21	30	43	113
6.2	21	21	21	21	20	29	40	115
6.3	22	21	21	21	21	30	41	115
6.4	22	22	21	21	21	29	42	114
6.5	21	21	22	21	21	30	42	113
A.1	40	39	40	39	39	82	153	549
A.2	41	40	40	39	39	84	157	543
A.3	40	40	40	39	39	82	155	544
A.4	38	38	38	38	38	85	157	549
A.5	40	39	40	39	39	82	154	542
B.1	22	22	22	22	22	84	125	960
B.2	22	22	22	22	22	83	123	955
B.3	23	22	22	22	22	88	128	964
B.4	23	23	23	22	22	84	127	942
B.5	23	23	22	22	22	83	125	938
C.1	44	44	44	43	43	122	228	1074
C.2	45	44	45	44	44	125	232	1079
C.3	45	44	45	44	44	122	231	1076
C.4	45	44	45	44	44	123	230	1074
C.5	45	44	45	44	44	122	228	1032
D.1	26	26	25	25	25	135	198	2468
D.2	25	25	25	25	25	137	200	2456
D.3	26	26	25	25	25	135	198	2358
D.4	26	26	25	25	25	136	199	2435
D.5	26	26	25	25	25	136	197	2446
E.1	5	5	5	5	5	15	17	54
E.2	5	5	5	5	5	15	16	55
E.3	5	5	5	5	5	15	16	51
E.4	5	5	5	5	5	17	18	55
E.5	5	5	5	5	5	15	16	56
NRE.1	17	17	17	17	17	256	309	20 373
NRE.2	17	17	17	17	17	256	309	20 381
NRE.3	17	17	17	17	17	257	310	20 372
NRE.4	17	17	17	17	17	256	311	20 376
NRE.5	17	17	17	17	17	258	312	20 368
NRF.1	10	10	10	10	10	416	449	41 960

Table 4
(continued)

Inst.	R-Gr 10 ³ it.	R-Gr 10 ⁵ it.	GRASP 10 ³ it.	GRASP 10 ⁵ it.	GRASP +SAT	R-Gr (s)	GRASP (s)	GRASP+ SAT (s)
NRF.2	11	11	10	10	10	421	452	41 521
NRF.3	11	11	10	10	10	416	449	41 856
NRF.4	11	11	10	10	10	419	451	41 450
NRF.5	11	11	10	10	10	418	449	42 076
CYC.6	63	60	62	61	60	44	35	6
CYC.7	156	156	158	155	144	162	130	26
CYC.8	379	378	382	377	348	699	600	645
CYC.9	900	894	898	888	813	1103	1132	442
CYC.10	2061	2061	2070	2063	1918	1805	1860	1922
CYC.11	4701	4688	4705	4677	4268	5550	7911	42 516
CLR.10-4	28	25	27	26	25	39	40	19
CLR.11-4	27	25	27	24	23	80	78	250
CLR.12-4	26	23	26	25	23	193	186	572
CLR.13-4	30	26	29	26	23	432	432	4987

The best solution found for the Random Greedy algorithm, column R-Gr, and the GRASP algorithm with $\alpha = 0.9$ and a local search based on redundancy testing, column GRASP, each one for 1000 and 100 000 iterations are reported. The solution of the GRASP with a SAT-based local search, column GRASP+SAT, is also given. Finally, the running times in seconds for each algorithm are also reported, values rounded to the nearest integer. Times correspond to 100 000 iterations R-Gr and GRASP and 500 iterations and MAXFLIPS = $10 \cdot |W|$ for GRASP+SAT.

A final remark is given for the CYC instances. The only instance where our algorithm does not improve the best previous solution, instance CYC.9, the best solution was found using a greedy heuristic with lexicographic resolution of ties. We must remark the difference between the best solution found using random resolution of ties highly differs from the solution obtained using the lexicographic resolution of ties. This phenomenon appears for all instances of the CYC category, leading us to believe that this effect is due to the instance generator used to create the files, generated these instances with a desirable ordering of variables for resolution of ties during a greedy construction procedure.

Acknowledgements

This work was partly funded by Grants BEC2003-03809 and DPI2004-03475 from the Spanish Government. We thank Nissan Spain and the UPC Nissan Chair for their help in this research work. Finally, we greatly acknowledge the substantial suggestions received by the referee of the paper.

References

- [1] Bautista J, Pereira J. Locating collection areas for municipal waste management: an application to the metropolitan area of Barcelona. OMEGA, in Press. Corrected proof, available online 2 March 2005.
- [2] Caprara A, Fischetti M, Toth P. A heuristic method for the set covering problem. *Operational Research* 1999;47(5):730–43.
- [3] Toregas C, Swain R, ReVelle C, Bergman L. The location of emergency service facilities. *Operational Research* 1971;19:1363–73.
- [4] Garey MR, Johnson DS. *Computers and intractability: a guide to the theory of NP-completeness*. New York: W.H. Freeman & Company; 1979.
- [5] Beasley JE. OR-Library: distributing test problems by electronic mail. *Journal of Operational Research Society* 1990;41(11):1069–72.
- [6] Fisher ML, Kedia P. Optimal solutions of set covering/partitioning problems using dual heuristics. *Management Science* 1990;36(6):674–88.
- [7] Beasley JE, Jornsten K. Enhancing an algorithm for set covering problems. *European Journal of Operational Research* 1992;58(2):293–300.
- [8] Balas E, Carrera MC. A dynamic subgradient-based branch-and-bound procedure for set covering. *Operational Research* 1996;44(6):875–90.
- [9] Beasley JE. A Lagrangean heuristic for set covering problems. *Naval Research Logistics* 1990;37(1):151–64.
- [10] Lopes FB, Lorena LA. Surrogate heuristic for set covering problems. *European Journal of Operational Research* 1994;79(1):138–50.
- [11] Beasley JE, Chu PC. A genetic algorithm for the set covering problem. *European Journal of Operational Research* 1996;94(2):392–404.
- [12] Solar M, Parada V, Urrutia R. A parallel genetic algorithm to solve the set-covering problem. *Computer Operational Research* 2002;29(9): 1221–35.
- [13] Lessing L, Dumitrescu I, Stützle T. A comparison between ACO algorithms for the set covering problem. *Lecture Notes in Computer Science* 2004;3172:1–12.

- [14] Yagiura M, Kishida M, Ibaraki T. A 3-flip neighborhood local search for the set covering problem. Technical Report #2004-001. Department of Applied Mathematics and Physics, Graduate School of Informatics, Kyoto University, 2004. Available at: <http://www-or.amp.i.kyoto-u.ac.jp/~yagiura/papers-e.html>.
- [15] Festa P, Resende MGC. An annotated bibliography of GRASP. Available at: <http://www.research.att.com/~mgr/papers.html>.
- [16] Almiñana M, Pastor JT. An adaptation of SH heuristic to the location set covering problem. *European Journal of Operational Research* 1997;100(3):586–93.
- [17] Grossman T, Wool A. Computational experience with approximation algorithms for the set covering problem. *European Journal of Operational Research* 1997;101(1):81–92.
- [18] Papadimitriou CH. Computational complexity. Reading, MA: Addison-Wesley; 1993.
- [19] Yagiura M, Ibaraki T. Efficient 2 and 3-flip neighborhood search algorithms for the MAX SAT: experimental evaluation. *Journal of Heuristics* 2001;7(5):423–42.
- [20] Resende MGC, Feo TA. A GRASP for satisfiability. In: Johnson DS, Trick MA, editors. Cliques, coloring, and satisfiability: second DIMACS implementation challenge. DIMACS series on discrete mathematics and theoretical computer science, vol. 260. Providence, RI: American Mathematical Society; 1996. p. 499–520.
- [21] Resende MGC, Pitsoulis LS, Pardalos PM. Approximate solution of weighted MAX-SAT problems using GRASP. In: Du D, Gu J, Pardalos PM, editors. DIMACS series on discrete mathematics and theoretical computer science satisfiability problem: theory and applications, vol. 35. Providence, RI: American Mathematical Society; 1997. p. 393–405.
- [22] Gu J. Efficient local search for very large-scale satisfiability problems. *ACM SIGART Bulletin* 1992; (3):8–12.
- [23] Hoos HH, Stutzle T. Local search algorithms for SAT: an empirical evaluation. *Journal of Automated Reasoning* 2000;24(4):421–81.
- [24] Mills P, Tsang EPK. Guided local search applied to the satisfiability (SAT) problem. In: Proceedings of the 15th national conference of the Australian Society for Operations Research (ASOR'99). Queensland, Australia, July 1999. p. 872–83.
- [25] Selman B, Levesque H, Mitchell D. A new method for solving hard satisfiability problems. In: Proceedings of the 10th national conference on artificial intelligence. Menlo Park, CA, USA: AAAI Press/The MIT Press; 1992. p. 440–6.
- [26] Selman B, Levesque H, Mitchell D. A new method for solving hard satisfiability problems. In: Proceedings of the 10th national conference on artificial intelligence. Menlo Park, USA; CA: AAAI Press: The MIT Press; 1992. p. 440–6.
- [27] Glover F, Laguna M. Tabu search. Dordrecht: Kluwer Academic Publishing; 1997.
- [28] Resende MGC, Ribeiro CC. Greedy randomized adaptive search procedures. In: Glover F, Kochenberger G, editors. Handbook of Metaheuristics. Dordrecht: Kluwer Academic Publishers; 2003. p. 219–49.
- [29] Laguna M, Martí R. Grasp and path relinking for 2-layer straight line crossing minimization. *INFORMS Journal of Computation* 2000;11(1): 44–52.
- [30] Prais M, Ribeiro CC. Reactive GRASP: an application to a matrix decomposition problem in TDMA traffic assignment. *INFORMS Journal of Computation* 2000;12(3):164–76.